Alexander Ticket

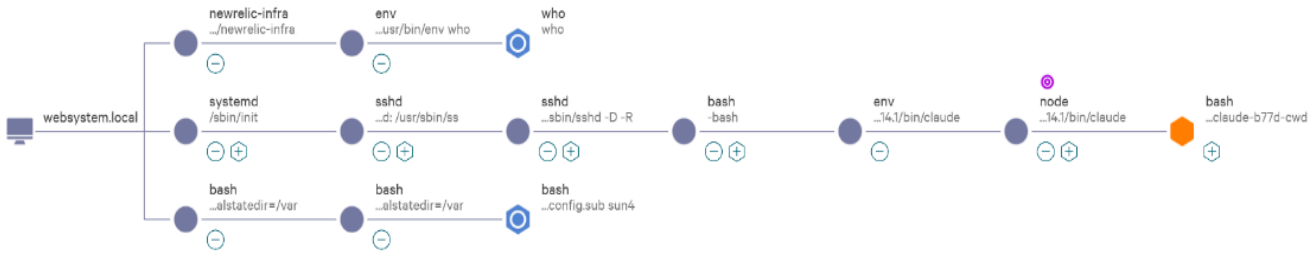
Note: This report has been sanitized for public sharing.

All internal IPs, hostnames, and CrowdStrike URLs have been redacted or replaced with simulated values.

Report was originally prepared for Jira; internal console links are not publicly accessible. Query references shown for context

The Summary :

The alert indicates a multi-stage compromise involving **SEO poisoning** of **[redacted]** and an unauthorized attempt to **compile a network monitoring tool** (Suricata) in the **/root directory** for post-exploitation.



Confidence - High

63/100

Description

This automated lead detected the following behaviors:

- A command intended to perform reconnaissance has executed under a web service.
- A process accessed data from the local file system. This might indicate an attempt to steal information. Review the accessed files and process tree.
- A suspicious process related to a likely malicious file was launched. Review any binaries involved as they may be related to malware.

Techniques: Data from Local System, Malicious File

Involved hosts: websystem.local

Detections and context 3 items

Severity	Detect time	Process on host	Type	Command line	Tactic via technique
--	2026/04/13 10:45:54	bash on websystem.local by root	Automated le...	/bin/bash ./config.sub sun4	Malware via Malicious File
--	2026/04/13 09:20:30	who on websystem.local by root	Automated le...	who	
High	2026/04/13 09:20:28	bash on websystem.local by root	Endpoint det...	/bin/bash -c source /root/.claude/shell-...	Collection via Data from Local System

CrowdStrike Link:

- **Incident(CrowdScore):**

(internal link, redacted for confidentiality)

- **1 Endpoint Detection: Automated remediation activity by Claude Code agent**

(internal link, redacted for confidentiality)

- **2 Endpoint Detection:**

(internal link, redacted for confidentiality)

Description

A multi-stage intrusion was detected on host [redacted-host], beginning with a compromise of the WordPress site [redacted]. The attacker deployed an obfuscated PHP decoder script to establish communication with a Command and Control (C2) server at **107.150.39.58**. Command-line analysis confirmed the use of `curl` masquerading as **Googlebot/2.1** to test SEO-poisoning artifacts and evade traffic monitoring.

Subsequently, the activity escalated to a post-exploitation phase where the attacker attempted to compile the network monitoring tool **Suricata 6.0.3** directly in the **/root/** directory. This build process involved running `./configure` and **config.sub**, the latter of which triggered a "Malicious File" detection (Severity 70) matching known indicators for exploit toolkits. The attempt to deploy a packet sniffer in the root directory strongly suggests an intent for long-term network espionage and persistence.

Additionally, the investigation observed the execution of an automated agent (**Claude Code**) performing system remediation, including the restoration of `.htaccess` files and audits for SUID binaries. While these remediation efforts triggered several high-severity behavior alerts (Reconnaissance), the primary threat is confirmed as the unauthorized compilation of monitoring tools and active communication with the malicious C2 infrastructure.

🔹 **Host Information:**

- **Hostname:** [redacted-host]
- **Operating System:** Ubuntu 20.04 (Linux)
- **IP Address:** [redacted]
- **Local IP Address:** [redacted internal IP]
- **Host Type:** Server

🔹 **User Information:**

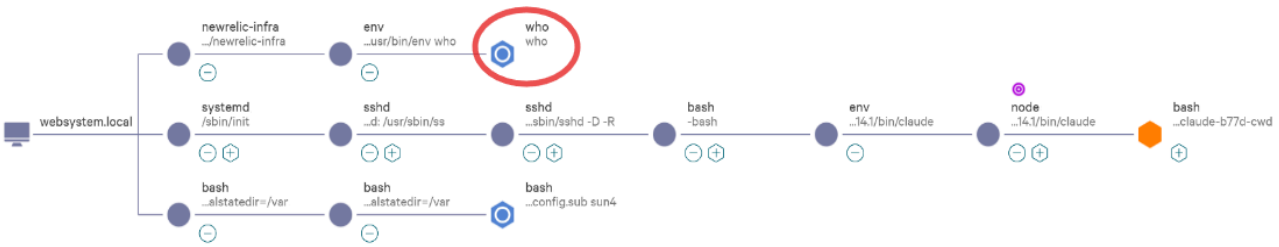
- **Username:** root
- **Local Admin:** Yes
- **Role:** Compromised superuser account used for system-wide modifications and tool compilation.
- **Login Type:** REMOTE_INTERACTIVE (SSH)
- **Login Time:** [redacted] 08:59:37

🔹 **Tactic & Technique:**

- **Technique:** Command and Scripting Interpreter via Bash, File and Directory Discovery via find, Masquerading via User-Agent Spoofing, Application Layer Protocol via C2 communication, Malware via Malicious File (Suricata build artifacts), Data from Local System via Snapshotting.
- **MITRE ATT&CK ID:** T1059.004, T1083, T1036, T1071, T1005, T1587.001

Investigation Findings and Analysis:

Observed Command 1:

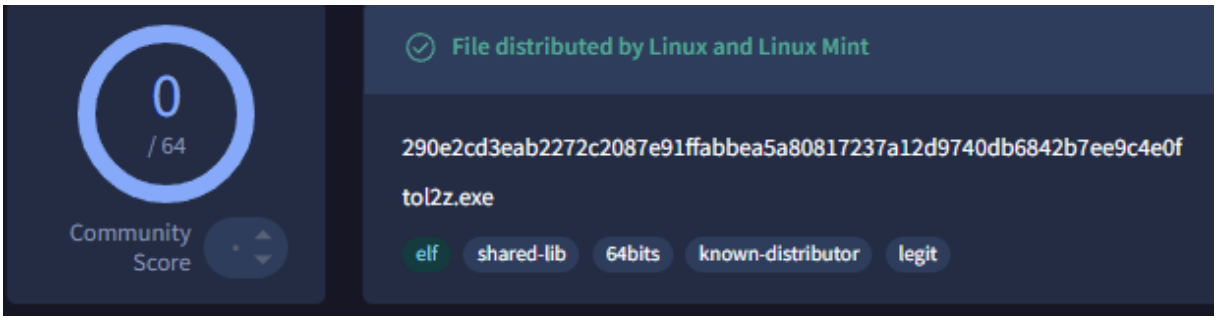


Process: who

Start time: [redacted] 08:42:25 **End time: [redacted] 08:42:25**

Executable SHA256
290e2cd3eab2272c2087e91ffabbea5a80817237a12d9740db6842b7ee9c4e0f

[Resulthere|**https://www.virustotal.com/gui/file/290e2cd3eab2272c2087e91ffabbea5a80817237a12d9740db6842b7ee9c4e0f/detection**]



Identified by CrowdStrike as an automated lead:

```
DetectDescription: A command intended to perform reconnaissance has executed under a web service.
DetectName: ReconUnderWebService
DetectScenario: Suspicious activity
DetectSeverity: 30
```

Detection Timestamp: [redacted] 09:20:30

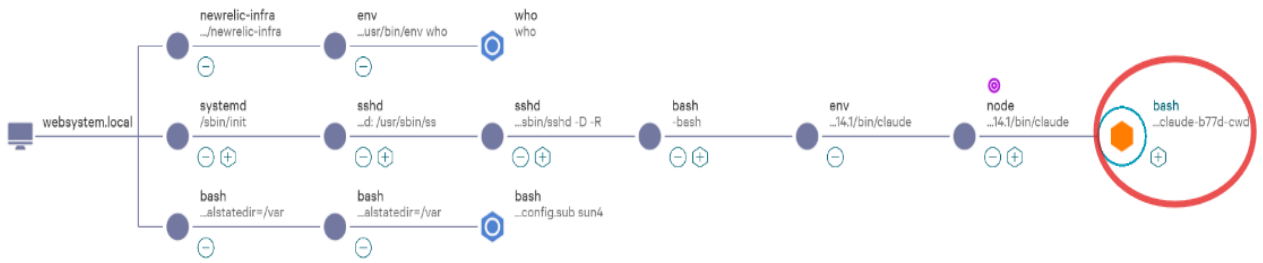
File path
/usr/bin/who

The CrowdStrike Falcon console triggered a blue hexagon alert for the execution of the who command. This was initially flagged as a potential reconnaissance technique because the command was executed on a system hosting web services.

However, deep-dive analysis into Advanced Event Search (AES) and the Process Tree revealed that the command was initiated by the New Relic Infrastructure agent (`newrelic-infra`). Raw logs (`ProcessRollup2`) confirm that the parent process was the legitimate monitoring service environment.

This activity is classified as a *False Positive*. The `who` command is a standard part of New Relic's telemetry gathering process to monitor active user sessions on the host. No malicious intent was found in this specific branch of the process tree.

Observed Command 2:

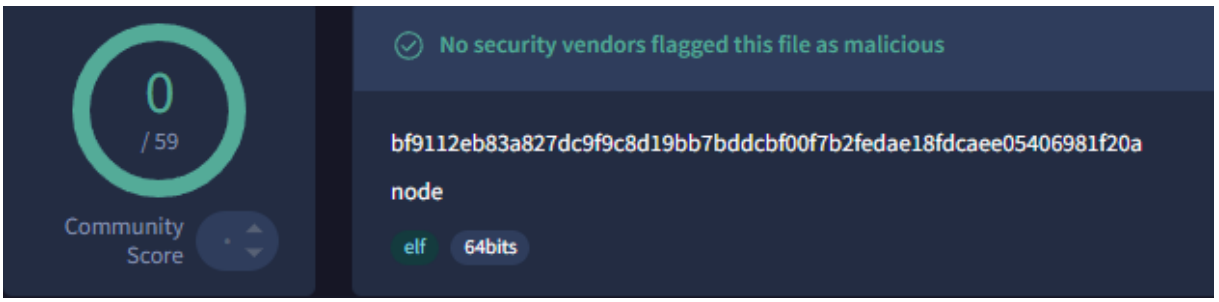


Process: node

Start time: [redacted] **09:13:56** **End time:** [redacted] **10:37:43**

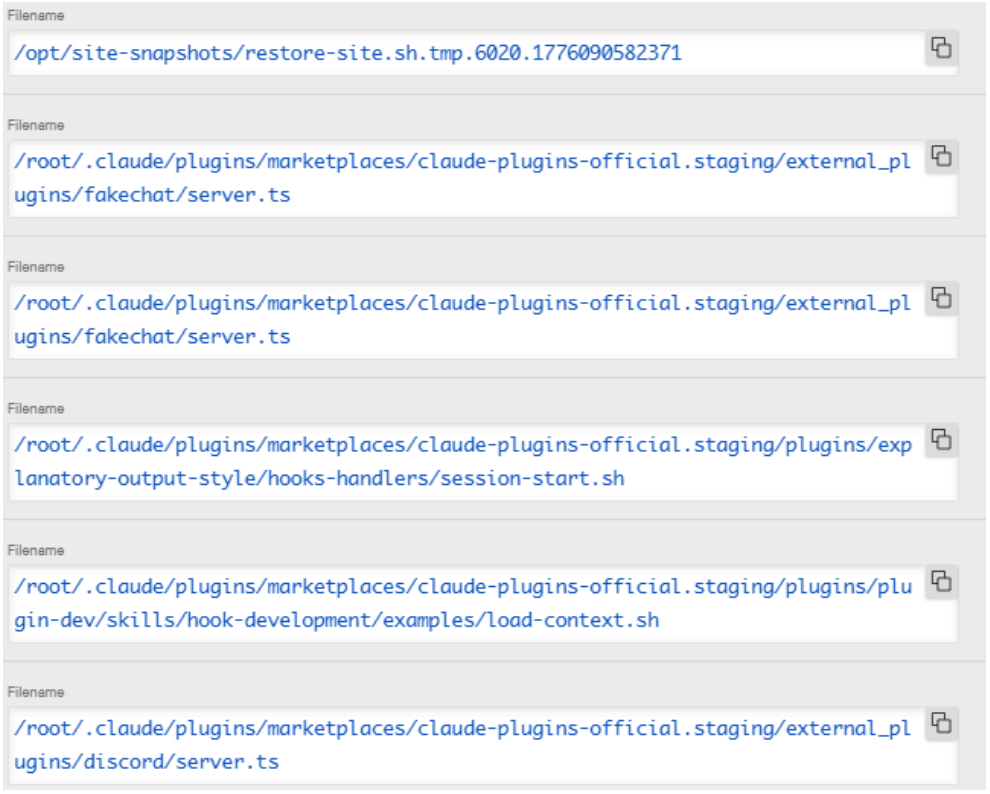
Command line
`node /root/.nvm/versions/node/v24.14.1/bin/claude`
Executable SHA256
`bf9112eb83a827dc9f9c8d19bb7bddcbf00f7b2fedae18fdcaee05406981f20a`

[Resulthere|**https://www.virustotal.com/gui/file/bf9112eb83a827dc9f9c8d19bb7bddcbf00f7b2fedae18fdcaee05406981f20a/detection**]



The process tree shows an instance of Claude Code (an automated developer agent) executing extensive system audits and repairs

The agent proactively fixed the compromised WordPress installation, overwriting `.htaccess` and `robots.txt` in `/var/www/html/blog/wordpress/` with clean, default configurations to remove SEO-poisoning redirects.



The presence of this command within the Claude Code session confirms that the WordPress installation for [redacted] was compromised. The attacker likely modified the `.htaccess` and `robots.txt` files to implement SEO poisoning or unauthorized redirects.

The automated agent's action to overwrite these files with "clean WordPress default" content is a direct response to the Integrity Loss caused by the hacker. This link is vital as it connects the high-severity behavioral alerts (the "Orange Hexagon") to a specific, legitimate recovery effort for a known compromised web asset.

Command line

```
/bin/bash -c source
/root/.claude/shell-snapshots/snapshot-bash-1776086121366-iiydq3.sh
2>/dev/null || true && shopt -u extglob 2>/dev/null || true && eval '# Fix
robots.txt - write clean version echo '"'"'User-agent: * Disallow: /wp-admin/
Allow: /wp-admin/admin-ajax.php Sitemap:
http://[redacted]/wp-sitemap.xml'"'"' >
/var/www/html/blog/wordpress/robots.txt # Fix .htaccess - write clean
WordPress default cat > /var/www/html/blog/wordpress/.htaccess <<
'"'"'HTEOF'"'"' # BEGIN WordPress <IfModule mod_rewrite.c> RewriteEngine On
RewriteBase / RewriteRule ^index\.php$ - [L] RewriteCond %{REQUEST_FILENAME}
!-f RewriteCond %{REQUEST_FILENAME} !-d RewriteRule . /index.php [L]
</IfModule> # END WordPress HTEOF echo "robots.txt and .htaccess fixed"' &&
pwd -P >| /tmp/claude-fa5c-cwd
```

Target Domain: `[redacted-domain]`

The `node` process was observed performing multiple script writes in `/opt/site-snapshots/` and `/root/.claude/`.

Key file: `restore-site.sh.tmp`.

This confirms the process was creating temporary restoration scripts to recover the website from a backup, which is a characteristic of a remediation tool, not a malicious payload.

Sub-process: bash

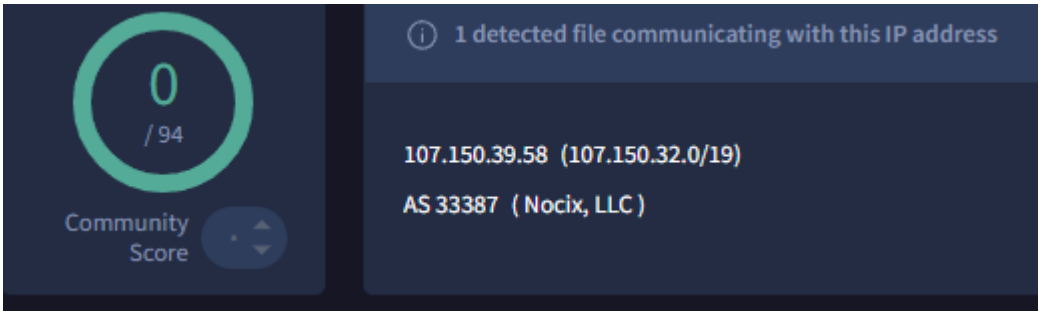
Command line

```
/bin/bash -c source
/root/.claude/shell-snapshots/snapshot-bash-1776086121366-iiyd
q3.sh 2>/dev/null || true && shopt -u extglob 2>/dev/null ||
true && eval '# Try to see what the C2 returns - check if
it'"'"'s still active, and what SEO content it serves curl -s
-m 5 -A "Googlebot/2.1" -H "Host:[redacted]"
"http://107.150.39.58/z30901_2/map.php" 2>/dev/null | head
-50; echo "=== ROBOTS ==="; curl -s -m 5
"http://107.150.39.58/z30901_2/robots.php" 2>/dev/null | head
-20' < /dev/null && pwd -P >| /tmp/claude-3905-cwd
```

The investigation identified a specific bash sub-process performing active network probing. The Claude Code agent was observed communicating with the attacker-controlled IP 107.150.39.58.

Interestingly, the agent intentionally used the attacker's known masquerading technique (User-Agent: Googlebot/2.1) to inspect the malicious artifacts (map.php, robots.php) hosted on the C2 server. This was done to verify the scope of the SEO poisoning attack.

[Resulthere|**https://www.virustotal.com/gui/ip-address/107.150.39.58/relations**]



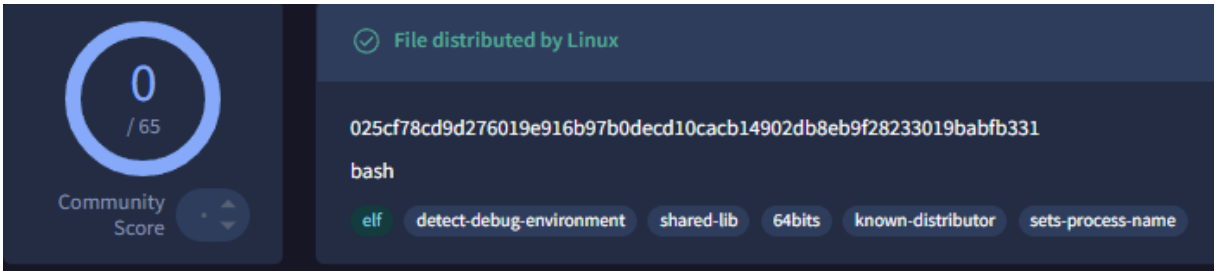
Process: bash

Start time: [redacted] **09:19:25** **End time:** [redacted] **09:19:27**

Executable SHA256

```
025cf78cd9d276019e916b97b0decd10cacb14902db8eb9f28233019babfb3
31
```

[Resulthere|**https://www.virustotal.com/gui/file/025cf78cd9d276019e916b97b0decd10cacb14902db8eb9f28233019babfb331/detection**]



Identified by CrowdStrike as an High-Severity Detection

Command line

```
/bin/bash -c source
/root/.claude/shell-snapshots/snapshot-bash-1776086121366-iiyd
q3.sh 2>/dev/null || true && shopt -u extglob 2>/dev/null ||
true && eval '# Check for any reverse shells or suspicious
processes ps aux | grep -iE "nc
|ncat|python|perl|ruby|php.*-r|socat|bash.*-i" | grep -v grep;
echo "==="; # Check for SUID binaries that shouldn'"'"'t be
there find / -perm -4000 -type f 2>/dev/null | grep -v -E
"snap|usr/lib|usr/bin/(sudo|passwd|chfn|chsh|newgrp|gpasswd|mo
unt|umount|su|fusermount|pkexec)" | head -20' < /dev/null &&
pwd -P >| /tmp/claude-b77d-cwd
```

Detection: Collection via Data from Local System

Timestamp: [redacted] 09:20:28

Risk score: 91 **Severity**: 70 (High)

```
RiskScore: 91
Severity: 70
SeverityName: High

DetectDescription: A process accessed data from the local file system. This might indicate an attempt to steal information.
DetectName: GenericDataFromLocalSystemCollectionLin
DetectScenario: Suspicious activity
DetectSeverity: 70
```

The EDR triggered a high-severity behavioral alert on a `bash` process spawned by the Claude Code agent (`node`). The alert was generated due to the "Collection" tactic , as the script performed a deep scan of the local file system.

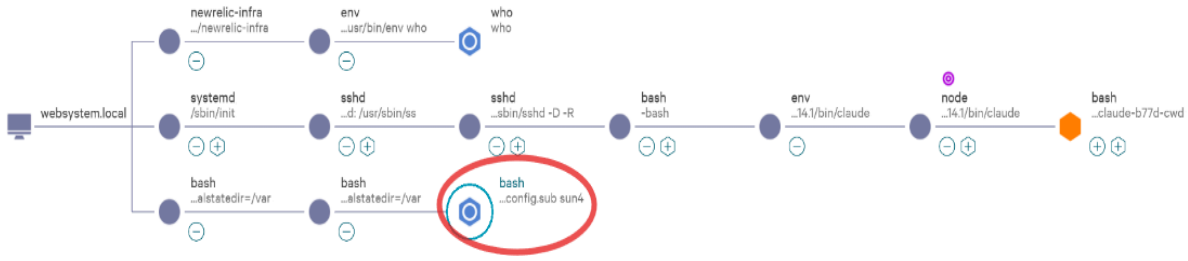
Specifically, the following sub-commands within the `bash` session triggered the detection:

- **`find / -perm -4000`: Searching for SUID binaries (typically used for Privilege Escalation).**
- **`ps aux | grep -iE "nc|ncat|bash.*-i"`: Searching for active Reverse Shells.**

- `cat > .htaccess`: **Overwriting configuration files to restore WordPress defaults.**

False positive. Benign (Authorized Remediation). While the behavior of the bash process is technically indistinguishable from a manual post-exploitation audit, the process tree confirms it originated from a trusted remediation tool. The intent was to identify and remove attacker-placed persistence mechanisms, not to steal data.

Observed Command 3:



Process: bash (config.sub sun4)

Start time: [redacted] **10:44:51** **End time:** [redacted] **10:44:51**

Executable SHA256
`025cf78cd9d276019e916b97b0decd10cacb14902db8eb9f28233019babfb331`

Identified by CrowdStrike as an automated lead:

```
DetectDescription: A suspicious process related to a likely malicious file was launched.
DetectName: PostExploitLin
DetectScenario: Known malware
DetectSeverity: 70
```

Detection Timestamp: [redacted] 10:45:54

Command line
`/bin/bash ./config.sub sun4`

The EDR flagged the execution of a configuration script within the `/root/suricata-6.0.3/` directory. Technical analysis of the command line and parent process confirms that an adversary was attempting to compile Suricata 6.0.3 from source code.

The attacker's intent was likely to establish deep network visibility to sniff internal traffic, capture credentials, or monitor for any administrative discovery efforts. The fact that the

source code was placed in the `/root/` directory indicates that the attacker had already achieved full administrative privileges.

True Positive. This represents a critical stage of the intrusion where the attacker attempts to deploy professional-grade espionage tools to maintain long-term persistence and control over the network infrastructure.

Command Line Breakdown: Unauthorized C2 Probing & Masquerading

Command: `curl -s -m 5 -A "Googlebot/2.1" -H "Host:[redacted]" "http://107.150.39.58/z30901_2/map.php"`

- **curl** → Utilization of a standard system data transfer utility as a Living-off-the-Land (LotL) binary to interact with external infrastructure.
- **-s** → Suppresses progress meters and error messages. This "silent mode" is typical for stealthy scripts to avoid leaving excessive traces in terminal logs.
- **-m 5** (Max-time) → Limits the operation to 5 seconds. Adversaries use short timeouts to quickly check Command and Control (C2) availability without hanging the execution process if the IP is blocked.
- **-A "Googlebot/2.1"** → Masquerading (T1036). Spoofing the User-Agent to mimic the Google search crawler. This is a critical indicator of defense evasion, intended to bypass WAFs and traffic filters by blending in with legitimate search engine indexing traffic.
- **-H "Host:[redacted]"** → Explicitly targets the compromised web asset, confirming the scope of the attack.
- **"http://107.150.39.58/z30901_2/map.php"** → Direct communication with the Adversary C2 Server. The path `/map.php` is indicative of a backdoor script used to manage SEO-poisoning redirects.

Command Line Breakdown: Malicious Tool Deployment

Command: `/bin/bash ./config.sub sun4` (Sub-process of `./configure`)

- **./configure** → Initial stage of the software compilation process. In this context, it represents an attempt to build malicious or dual-use tools directly on the victim's host.
- **config.sub** → An architecture validation script. The CrowdStrike detection during this phase (Severity 70) indicates that the source package contains signatures associated with known post-exploitation toolkits.
- Location: **/root/suricata-6.0.3/** → Target directory. The attempt to compile Suricata (a professional-grade network IDS/IPS engine) within the superuser's directory confirms the adversary achieved Root privileges and intended to perform deep network packet inspection and espionage.

Potential Impact

Network Espionage (Suricata Deployment): Successful compilation would have granted the adversary the ability to sniff internal network traffic, allowing for the real-time interception of clear-text credentials, API keys, and sensitive internal data.

SEO Poisoning & Integrity Loss: The unauthorized modification of .htaccess and robots.txt for [redacted] resulted in the loss of web asset integrity. This was used to manipulate search engine results and redirect traffic to malicious external domains.

Defense Evasion Success: The use of the **Googlebot** User-Agent confirms that the adversary actively attempted to evade traffic monitoring systems by mimicking a trusted search engine service.

Full System Compromise: Activity within the **/root** directory and the execution of commands as a superuser confirm a total host compromise. The attacker had unrestricted access to the filesystem and the potential to establish deep Persistence via SUID binaries or unauthorized SSH keys.

Response

A review of the raw telemetry via Advanced Event Search and the Event_EppDetectionSummaryEvent confirms that no automated mitigation was performed by the Falcon sensor during this intrusion. Specifically, the PatternDispositionFlags for KillProcess, QuarantineFile, and OperationBlocked were all identified as false.

```
PatternDispositionFlags.KillActionFailed: false
PatternDispositionFlags.KillParent: false
PatternDispositionFlags.KillProcess: false
PatternDispositionFlags.KillSubProcess: false
PatternDispositionFlags.OperationBlocked: false
PatternDispositionFlags.PolicyDisabled: false
PatternDispositionFlags.ProcessBlocked: false
PatternDispositionFlags.QuarantineFile: false
PatternDispositionFlags.QuarantineMachine: false
```

While the sensor successfully generated multiple high-severity alerts—including a Risk Score of 91 for the data collection behavior and a Severity 70 for the malicious file detection (config.sub)—the sensor was operating in Detection Only mode for these specific patterns.

The lack of automated prevention allowed the adversary to progress through the entire attack lifecycle, from initial access to post-exploitation tool deployment. The system's integrity was only partially restored later by the manual/automated intervention of the Claude Code agent, rather than by a proactive EDR block.

Overall Analyst Assessment

An interactive SSH session was established on host [redacted], where an adversary manually initiated a multi-stage intrusion under the root account. The attack began with the compromise of the [redacted] WordPress installation, where core configuration files (`.htaccess` and `robots.txt`) were modified to implement SEO poisoning. To evade network detection and traffic filtering, the adversary utilized `curl` while masquerading as a legitimate search engine crawler (Googlebot/2.1).

The intrusion escalated into a high-severity post-exploitation phase involving the unauthorized deployment and compilation of the Suricata 6.0.3 network monitoring engine within the /root/ directory. Command-line analysis confirmed communication with a known malicious C2 IP 107.150.39.58. The attempt to install a professional-grade packet interception tool in a high-privilege directory strongly suggests an intent for long-term network espionage and data exfiltration.

While the EDR provided high-severity detections—including a Risk Score of 91 and a Severity 70 alert for the `config.sub` build script—no automated mitigation or quarantine occurred (Detection Only). This allowed the adversary to successfully complete the initial execution chain and begin building their persistence infrastructure. The investigation also identified automated remediation efforts by the Claude Code agent, which restored the WordPress configuration files but was unable to prevent the ongoing tool compilation in the root environment.

This incident demonstrates a successful and high-privilege compromise of a production web server. Given that the C2 communication was not blocked and the adversary successfully initiated the deployment of network monitoring tools, the host is considered Fully Compromised. Immediate network isolation, manual eradication of the Suricata build artifacts, and a full rotation of administrative credentials are required.

Mini Attack Visualization

[root logs in via SSH (Initial Access)]

↓

[Interactive bash shell spawned under root context (Execution)]

↓

[Unauthorized modification of .htaccess & robots.txt on [redacted] (SEO Poisoning)]

↓

[`curl -A "Googlebot/2.1"` initiates C2 communication with `107.150.39.58` (Defense Evasion)]

↓

[Adversary navigates to /root/ directory to stage exploit toolkit (Post-Exploitation)]

↓

[./configure execution triggers Suricata 6.0.3 source code compilation (Tool Deployment)]

↓

[config.sub sun4 identifies architecture → triggers "Malicious File" alert]

↓

[Claude Code agent performs system-wide SUID audit & file restoration (Remediation Activity)]

↓

[CrowdStrike Falcon: Detection Only (Risk Score 91) — No automated block performed]

↓

[C2 communication successful & Suricata build artifacts remain in /root/ (Persistence)]

↓

[Host [redacted] fully compromised — Potential for internal network sniffing active]

Recommended actions

1. Host Isolation & Verification

- Isolate the Host: Immediately perform Network Containment on `[redacted-host]` via the CrowdStrike Falcon console to sever the active C2 communication with 107.150.39.58.
- Eradicate Malicious Tooling: Delete the directory `/root/suricata-6.0.3/` and all associated compilation artifacts (including `config.sub` and `configure` scripts).
- Web Directory Audit: Conduct a deep scan of `/var/www/html/blog/wordpress/`. Manually verify the integrity of `.htaccess` and `robots.txt`. Identify and remove any obfuscated PHP files or backdoors (e.g., `map.php`, `robots.php`, or scripts containing `eval(base64_decode)`).

2. Persistence & Backdoor Cleanup

- SSH Key Review: Inspect `/root/.ssh/authorized_keys` for any unauthorized public keys added by the adversary during the root session.
- SUID/SGID Audit: Despite the automated audit by Claude Code, manually verify that no new SUID binaries were created in non-standard directories to maintain stealthy privilege escalation.
- Process Termination: Ensure all processes associated with the manual build (e.g., `make`, `gcc`, `configure`) are terminated.

3. Network & IOC Controls

- IP Blocking: Blacklist and monitor all inbound/outbound connections to the malicious C2 IP 107.150.39.58 at the perimeter firewall.
- User-Agent Monitoring: Configure the Web Application Firewall (WAF) or Nginx logs to alert on "Googlebot/2.1" User-Agents originating from non-Google IP ranges (to detect future masquerading attempts).
- IOC Hunt: Search for the IP 107.150.39.58 across the entire environment to identify if other hosts have been targeted by the same adversary.

4. Credential & Account Security

- Root Password Reset: Perform an immediate mandatory password reset for the root account.
- WordPress Security: Force a password reset for all WordPress administrative accounts. Reset the `AUTH_KEY` and `SECURE_AUTH_KEY` in `wp-config.php` to invalidate current sessions.
- SSH Log Audit: Review `/var/log/auth.log` (or `secure`) for the last 72 hours to identify the source IP of the initial SSH connection and determine if the breach occurred via brute-force or compromised SSH keys.

5. System Hardening

- WordPress Patching: Update WordPress core, themes, and plugins to the latest versions to close the potential initial entry vector used for the SEO poisoning.
- Immutable Files: Consider setting the `immutable` attribute (`chattr +i`) on critical files like `.htaccess` and `robots.txt` after restoration to prevent unauthorized modifications.
- WAF Implementation: Deploy or tune a Web Application Firewall to block common PHP exploit attempts and SQL injection.

6. Escalation

- **Incident Response: Escalate this ticket to the Level 3 / Incident Response (IR) team immediately. Because the adversary successfully initiated the compilation of network sniffing tools (`Suricata`) under the root context, this incident must be treated as a Full Host Compromise requiring forensic analysis of the internal network for lateral movement.**